PS 53

Rock Paper Scissors

A console game in Python 3: conditions, functions and random

Topics explained | Full code | Test cases

OVERVIEW

Problem statement

Build a console game where the player plays **3 rounds** of Rock, Paper, Scissors against the computer.

Show

the winner of every round

Keep

the score: wins, losses, draws

Declare

the final result of the match

Language: Python 3 **Concepts: Conditions, Functions, Random**

For testing, the computer's 3 choices are fixed as Scissors, Rock, Paper.

CONCEPTS

Topics used in this project

Conditions

if / elif / else decide who wins a round and the final match result.

Functions

Reusable blocks: computer choice, winner check, input, match, play again.

Random

The random module lets the computer pick rock, paper or scissors.

Loops

for repeats 3 rounds; while True repeats input until it is valid.

Input validation

Reject anything other than rock, paper, scissors (or y / n).

Score keeping

A dictionary stores player, computer and draw counts.

HOW IT WORKS

Game rules

Player	Computer	Winner
rock	scissors	Player
scissors	paper	Player
paper	rock	Player
same choice	same choice	Draw
anything else	any	Computer

Rock beats Scissors

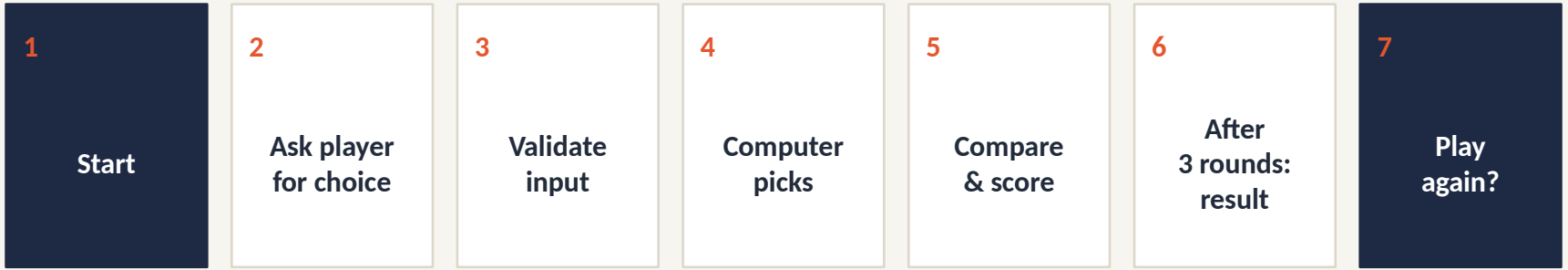
Scissors beats Paper

Paper beats Rock

The table's last row is the computer-wins case: whenever the player does not win or draw, the computer wins the round.

ALGORITHM

Program flow



Steps 2-5 repeat for each of the 3 rounds (for loop). Invalid input at step 3 repeats the question without losing the round (while loop). If the player answers y at step 7, the whole match starts again.

CODE 1 - RANDOM

Setup and computer's choice

```
import random

CHOICES = ["rock", "paper", "scissors"]
ROUNDS = 3
LABELS = {"player": "You win",
          "computer": "Computer wins",
          "draw": "Draw"}

TEST_MODE = True    # True = fixed computer moves
TEST_MOVES = ["scissors", "rock", "paper"]

def get_computer_choice(round_no):
    if TEST_MODE:
        return TEST_MOVES[round_no - 1]
    return random.choice(CHOICES)        # random module
```

Key points

- import random gives access to random.choice().
- CHOICES is the list of valid moves.
- TEST_MODE = True makes the computer play Scissors, Rock, Paper for the test cases.
- Set TEST_MODE = False for a real random game.

Comparing two choices

```
def decide_winner(player, computer):
    if player == computer:
        return "draw"
    if (player == "rock" and computer == "scissors") or \
       (player == "scissors" and computer == "paper") or \
       (player == "paper" and computer == "rock"):
        return "player"
    return "computer"
```

Key points

- A function that takes both choices and returns the winner.
- Equal choices give a draw.
- Three winning combinations for the player are checked with if / or.
- Everything else means the computer wins.

CODE 3 - LOOP AND VALIDATION

Validating the player's input

```
def get_player_choice(round_no):
    while True:
        prompt = f"Round {round_no} - rock, paper or scissors? "
        choice = input(prompt).strip().lower()
        if choice in CHOICES:
            return choice
        print("Invalid choice. Enter rock, paper or scissors.")
```

Key points

- while True keeps asking until the input is valid.
- .strip().lower() accepts ' Rock ' and 'ROCK'.
- Invalid input prints the error and asks again, so no round is lost.

CODE 4 - LOOP, SCORE, RESULT

Playing the three rounds

```
def play_match():
    scores = {"player": 0, "computer": 0, "draw": 0}
    summary = []
    for round_no in range(1, ROUNDS + 1):
        player = get_player_choice(round_no)
        computer = get_computer_choice(round_no)
        print(f"You: {player} | Computer: {computer}")
        winner = decide_winner(player, computer)
        scores[winner] += 1
        summary.append(f"Round {round_no}: {LABELS[winner]}")

    print(" | ".join(summary))
    print(f"You {scores['player']} | Computer {scores['computer']}"
          f" | Draws {scores['draw']}")
    if scores["player"] > scores["computer"]:
        print("Result: You win the match!")
    elif scores["player"] < scores["computer"]:
        print("Result: Computer wins the match.")
    else:
        print("Result: Match drawn.")
```

Key points

- for loop runs 3 rounds.
- scores dictionary counts wins, losses and draws.
- summary list is joined with ' | ' to print the round line.
- Final if / elif / else declares the match result.

CODE 5 - PUTTING IT TOGETHER

Play again and main program

```
def ask_play_again():
    while True:
        answer = input("Play again? (y/n): ").strip().lower()
        if answer in ("y", "n"):
            return answer == "y"
        print("Invalid choice. Enter y or n.")

def main():
    print("=== Rock Paper Scissors ===")
    while True:
        play_match()
        if not ask_play_again():
            print("Thanks for playing!")
            break

if __name__ == "__main__":
    main()
```

Key points

- ask_play_again() accepts only y or n.
- main() repeats play_match() until the player answers n.
- __name__ check lets the file run from the console.
- Save as rock_paper_scissors.py and run: python rock_paper_scissors.py

TESTING

Test cases and results

Case	Player input	Expected output (computer: Scissors, Rock, Paper)
1 Normal	rock, rock, rock	You win | Draw | Computer wins You 1 | Computer 1 | Draws 1 Match drawn.
2 Boundary	rock, paper, scissors	You win x3 You 3 | Computer 0 | Draws 0 You win the match!
3 Invalid	stone, then rock	Invalid choice. Enter rock, paper or scissors. Round 1: You win (no round lost)
4 Different	scissors, scissors, rock Play again? n	Draw | Computer wins | Computer wins You 0 | Computer 2 | Draws 1 Computer wins the match. Thanks for playing!

All 4 cases were run against the code and match the expected output.

CHECKLIST

Bare minimum requirements covered

Python 3, one .py file, run from console	`rock_paper_scissors.py`
random module for the computer's choice	`random.choice(CHOICES)`
Function comparing choices with conditions	`decide_winner()`
Loop for 3 rounds, score, match result	`play_match()`
Validate input and ask 'Play again? (y/n)'	`get_player_choice()`, `ask_play_again()`

Conclusion and submission

- The game uses conditions, functions, random, loops and validation in a small, readable program.
- All 4 test cases give the expected output.
- Next: write the report, upload the code and report to GitHub, and submit before the deadline.

Thank you